EDGE AI & THE RASPBERRY PI

Reading code at the bottom of a page, e.g. **B-20-21** = read the Edge AI book, PDF pages 20-21.

1. By the end of today I can...

- [ ] explain Edge AI in two sentences
- [ ] name the main parts of the Pi 4
- [ ] write the OS onto a microSD card with my own settings
- [ ] power the Pi, read its 2 LEDs, wait out first boot
- [ ] log in over **SSH** from Windows and shut down safely

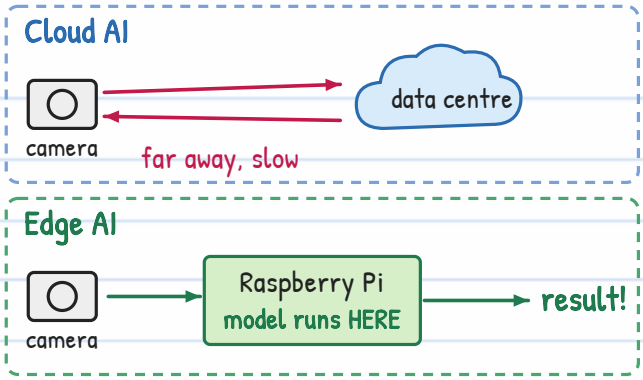
My 2 hours

A big picture	20 min
B meet the board	20 min
C prepare microSD	45 min
D first boot	15 min
E first SSH login	20 min

my notes

2. What is Edge AI?

- **Cloud AI:** my data travels to a data centre and the answer travels back.
- **Edge AI:** the AI algorithm runs locally, on the device next to the data (camera, robot, sensor box, a Pi) - not in the cloud.
- Result: devices that are more **responsive, private and efficient**.
- Two kinds of AI I will build on the Pi: **fixed-function** (reactive) and **generative** (proactive).



my notes

3. Five reasons it matters

Advantage	What it means in practice
Low latency	answer in milliseconds, no network round trip (a robot reacts *now*)
Privacy	sensitive images / audio stay on the device, not on a remote server
Network independence	keeps working with no Internet: tunnel, field, factory
Less bandwidth	send only the useful result, not a whole video stream
Energy efficiency	optimised models on small hardware use little power

In one sentence: Edge AI brings intelligence to where data is generated and decisions are needed.

my notes

B-20-21, 27

HOW THE COURSE FITS TOGETHER

4. What I will learn (the whole path)

1. **Essential setup:** get the Pi ready for Edge AI work.
2. **Computer vision:** image classification and object detection.
3. **Small language models (SLMs):** run and optimise them on the Pi.
4. **Vision-language models:** multimodal AI (Florence-2).
5. **Physical computing:** connect AI to sensors and actuators.
6. **Advanced optimisation:** RAG, agents, function calling.

my notes

5. Two families of AI

Fixed-function (reactive)

- analyses a specific input using predetermined patterns / rules
- acts only when a particular trigger arrives
- consistent, predictable output
- excels at structured tasks: classify, detect, recognise
- works within its boundaries

my Phase 2: classification, detection, counting

Generative (proactive)

- creates new content: text, descriptions, solutions
- understands context
- keeps a dialogue going
- adapts to new situations beyond its training

my Phase 3: SLMs, vision-language, agents

	Fixed-function	Generative
Core focus	data analysis, assessment, automation	content creation, anticipation, dialogue
Examples	fraud detection, spam filters, image recognition	ChatGPT, DALL-E, predictive maintenance, smart assistants
Typical use	banking, medical diagnostics, security	content, customer support, design, automation

Rule of thumb: fixed-function = efficient, predictable, low resources, well-defined tasks. Generative = creative, personalised, anticipatory.

my notes

6. How each topic is taught, and what I should know

Every chapter has:

- concepts and theory
- step-by-step implementation
- a practical project with code
- performance optimisation tips
- ideas for further exploration

Helpful background:

- basic Python
- Linux command-line basics
- elementary machine-learning ideas
- earlier Pi experience (helpful, not required)

my notes

B-21-23, 25-27

WHY THE RASPBERRY PI?

7. What makes the Pi good for engineers

- **Computational power:** multi-core ARM processors, up to 8 GB RAM on the newest models.
- **GPIO:** a 40-pin header to talk directly to sensors and actuators.
- **Connectivity:** Wi-Fi, Bluetooth, Ethernet, several USB ports.
- **Low-level access:** I2C, SPI and UART interfaces.
- **Soft real-time:** with proper configuration, usable for control and signal processing.
- **Power efficiency:** low consumption, good for battery-powered designs.

Plus: a full Linux OS, a huge community, and low cost. That combination is why it is a good Edge AI platform.

my notes

8. Pi models compared

	Zero 2 W	Pi 4 (my board)	Pi 5
CPU	1 GHz quad-core Cortex-A53 (64-bit)	quad-core Cortex-A72, 1.5 GHz	quad-core Cortex-A76, 2.4 GHz
RAM	512 MB	4 GB (mine)	2 / 4 / 8 GB (8 GB best for AI)
Wi-Fi / BT	2.4 GHz only, BT 4.2	dual-band, BT 5.0	dual-band 802.11ac, BT 5.0
Ports	mini-HDMI, micro-USB OTG, CSI camera	2 micro-HDMI, 2 USB 3 + 2 USB 2, CSI, DSI	2 micro-HDMI, 2 USB 3 + 2 USB 2, CSI, DSI, PCIe
Power	5 V / 2.5 A micro-USB	5 V / 3 A USB-C	5 V / 5 A USB-C (27 W)
Typical	tiny embedded, ~0.6 W idle	vision projects, small SLMs	edge computing, vision, bigger SLMs (~3-3.5 W idle)

For me: everything in the vision phase runs well on the Pi 4. For language models I stay with small ones and expect them to be slower than on a Pi 5.

my notes

9. What I can build with a Pi

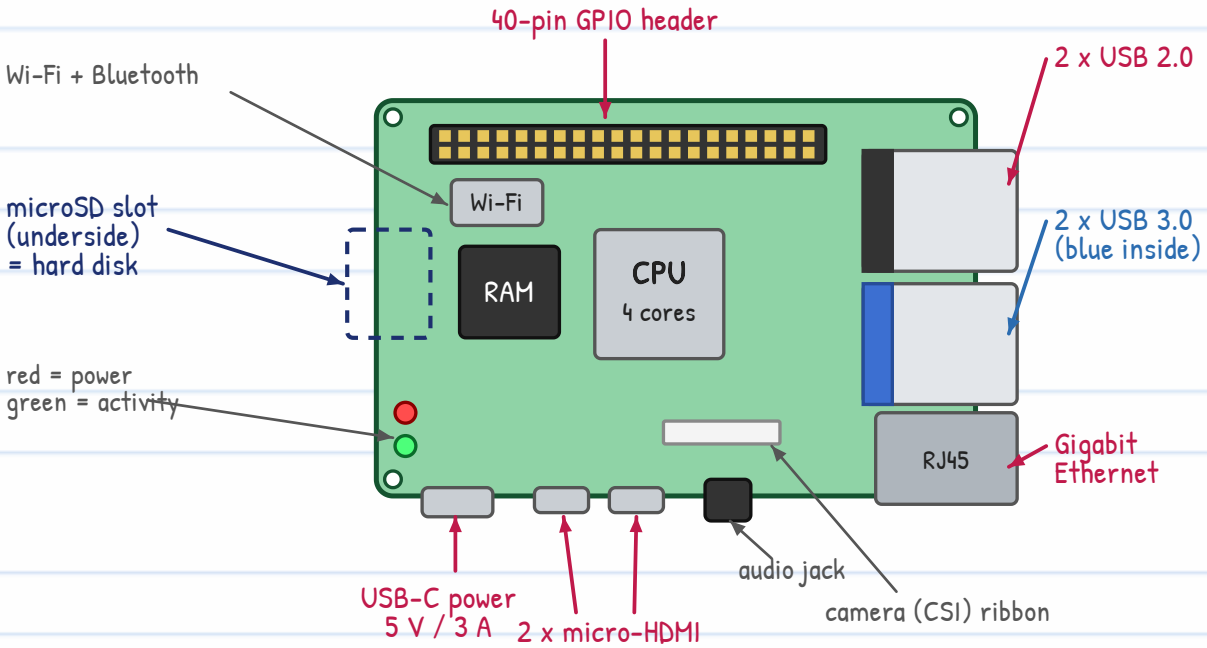
- **Embedded systems** design and prototyping
- **IoT** devices (MQTT, CoAP, HTTP/HTTPS)
- **Control systems** (feedback loops, PID, actuators)
- **Computer vision + AI** (OpenCV, TensorFlow Lite)
- **Data acquisition** and logging
- **Robotics** controllers, motion planning
- **Signal processing** and DSP
- **Network security** (VPNs, firewalls, testing)

my notes

B-29-32

MEET THE BOARD

10. Anatomy of the Raspberry Pi 4



(schematic - port positions simplified)

Part	What it does	Why I care
CPU, 4 cores	runs the OS + my code	does the neural-network maths
RAM 4 GB	working memory	models must fit in it
microSD	the "hard disk"	always shut down first
GPIO, 40 pins	sensors, LEDs, buttons	physical computing (Phase 3)
CSI / USB	camera ports	vision projects (Day 4)

The Pi 5 (not mine) adds a power button and a PCIe connector.

my notes

11. My turn

- [ ] find every part on my real board, take a top-down photo, label it
- [ ] hold the board by its edges - no metal table, no wet fingers

my notes

B-32-33

THE OPERATING SYSTEM

12. What is an operating system (OS)?

- Software that **manages the hardware and the software** and gives programs standard services. The layer between hardware and applications.

What it does

- **Processes:** shares CPU time between programs
- **Memory:** gives out and frees RAM
- **Files:** organises files and folders
- **Devices:** talks to connected hardware
- **User interface:** lets me use the computer

What it is made of

- **Kernel:** the core, controls the hardware
- **Shell:** where I type commands
- **File system:** how data is stored
- **Drivers:** let the OS talk to devices

my notes

13. Raspberry Pi OS

- A version of **Linux (Debian)** made for the Pi's ARM processor and limited resources. Formerly called Raspbian.
- **Lightweight**, **versatile** (many languages), **open source**, supports **GPIO**, regularly updated.
- Standard Linux tools and libraries work, so it is a full computer in a small package: IoT devices up to edge machine-learning projects.

Version note: the course text was written on Debian 12 "Bookworm". A newer Imager may offer a newer Debian. The steps are the same, but the **Python version** can differ, and TensorFlow Lite is picky - I record it today and check on Day 4-5.

my notes

14. Which OS image do I choose?

Choice in Imager	What it is	Mine?
Raspberry Pi OS (64-bit)	full system **with desktop**; 64-bit is needed by many AI libraries	YES
Raspberry Pi OS Lite (64-bit)	no desktop, saves RAM (used on tiny boards like the Zero)	not today
32-bit versions	older, less compatible	avoid

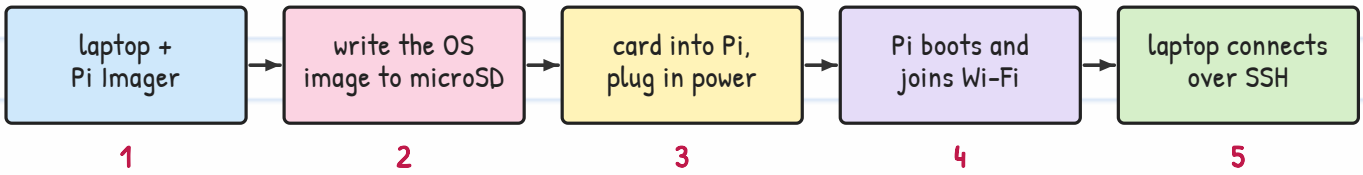
my notes

B-34-35

PREPARE THE MICROSD CARD

15. The idea

- By default a Pi looks for an operating system on the **microSD card** in its slot. It has **no hard disk**.
- "Installing the OS" = writing an image onto the card with **Raspberry Pi Imager** (Windows, macOS, Linux).
- Imager also pre-sets my **credentials and remote-access settings**: no monitor or keyboard needed = **headless**.



my notes

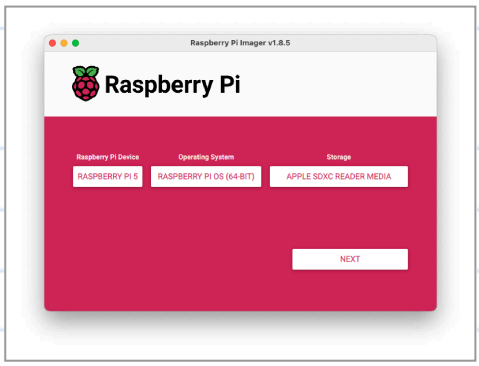
16. Before I start

- [ ] Raspberry Pi Imager installed (raspberrypi.com/software)
- [ ] microSD card, 32 GB recommended, in a reader
- [ ] Wi-Fi name + password ready
- [ ] card backed up - Imager **erases** it
- [ ] other USB drives unplugged (so I cannot pick them by mistake)

my notes

17. Steps on Windows

1. Open Imager. If Windows says "format the disk?" -> **Cancel**.
2. Device = Raspberry Pi 4
3. OS = Raspberry Pi OS (64-bit)
4. Storage = my card (check size + drive letter!)
5. Next -> open the settings (gear icon / "customise")
6. Write -> verify -> "Write successful" -> remove card



Imager with device, OS and storage chosen (example on a Mac, older release)

my notes

B-35-36

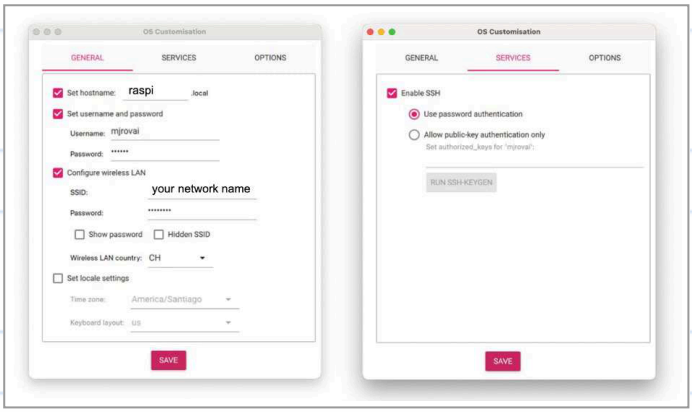
SETTINGS & FIRST BOOT

18. Settings I type (very important)

Setting	What to enter
hostname	`raspi4` (for my two Pis: `raspi-office` / `raspi-home`)
user + password	`piuser` + a strong password - write it on my connection card
Wi-Fi	name + password exactly as spelled, country `KW`
zone / keyboard	`Asia/Kuwait`, `us`
SSH	**ENABLE, password authentication** (Services tab) - without it I cannot log in remotely

Careful! Imager erases whatever drive I choose. Wrong drive = lost data.

Office Wi-Fi may need a login or block devices. Plan B: phone hotspot or an Ethernet cable. Imager stores one Wi-Fi network only; I add the second one on Day 2.



settings screens: General (left), Services (right)

my notes

19. First boot (15 min)

1. Card into the slot on the **underside** (contacts toward the board).
2. Optional monitor / keyboard first.
3. Power **last** (USB-C, 5 V / 3 A). The Pi 4 has **no power button**.
4. Wait a few minutes; it may restart once. **Do not unplug!**

- RED steady = power is good
- GREEN flickers = Pi reads the card

no red light? check the power supply

first boot: resizes the file system, applies my settings, starts SSH (1-3 min)

If something is off: nothing lights up = power; red only, green never flickers = no OS found, reseat or re-flash; random restarts = weak charger.

my notes

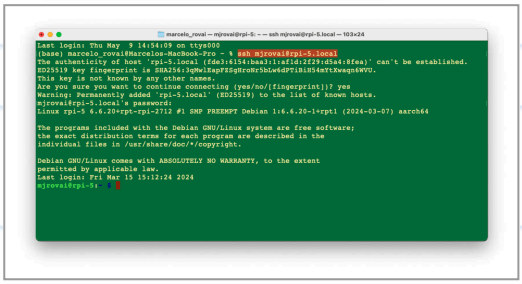
B-36-37

FIRST SSH LOGIN

20. Logging in from Windows (20 min)

- **SSH** = Secure SHell: the Pi's command line in a window on my laptop, over Wi-Fi, encrypted. Both machines on the **same network**.
- Windows tools: PowerShell or Command Prompt (older option: PuTTY). Mac/Linux: Terminal.
- I can use the IP address (`ssh piuser@192.168.x.x`) or the name (`ssh piuser@raspi4.local`).

1. Start -> **PowerShell**. Test: `ssh -V`
2. Connect (below). First time: type `yes`.
3. Password: **nothing shows** as I type - normal!
4. Prompt `piuser@raspi4:~ $` = I am **inside the Pi**.



first login (example on a Mac)

```
PS C:\> ssh piuser@raspi4.local
Are you sure you want to continue connecting (yes/no)? yes
password: (invisible)
piuser@raspi4:~ $ _
```

my notes

21. First commands, updates, shutdown

Command	What it does / what I should see
`hostname -I`	the Pi's IP address - write it down
`uname -m`	want **aarch64** (= 64-bit); `armv7l` means 32-bit
`cat /etc/os-release`	OS version - note it
`python3 --version`	needed on Day 4 (TFLite is picky)
`free -h`	memory: close to 4 GB
`sudo apt-get update` `sudo apt upgrade`	update the package list, then the system - do it regularly

ALWAYS shut down properly: `sudo shutdown -h now`
The Pi may still be writing to the card. Wait until the **LED stops blinking and goes dark**, then unplug. Pulling the plug can lose data or corrupt the card.

Moving files between laptop and Pi: pen drive, `scp` in the terminal, or an FTP/SFTP program (Day 2).

my notes

B-38-39

TROUBLESHOOTING & TWO PIS

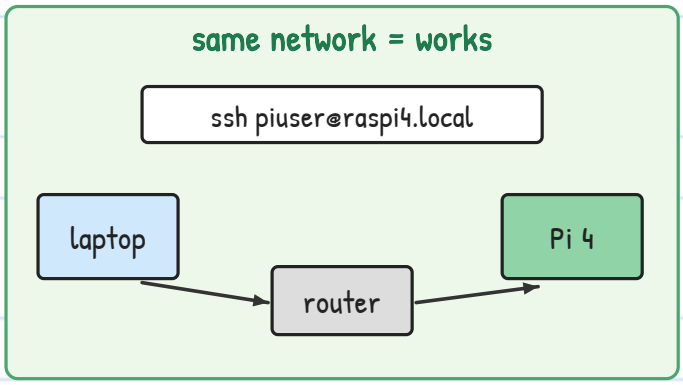
22. When it does not work

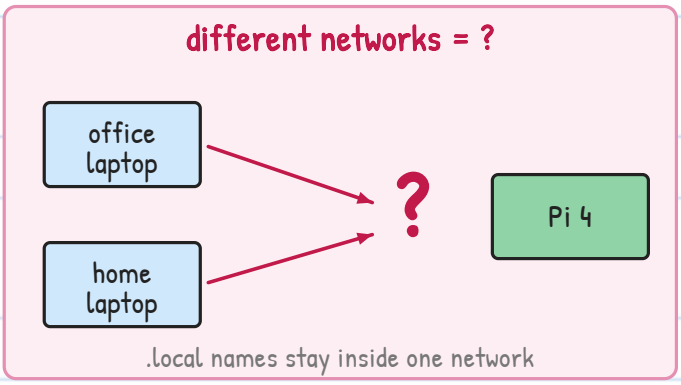
I see	probably	try
`Could not resolve hostname`	Pi not on the network yet, Wi-Fi typo, `.local` blocked	wait 5 min, check Wi-Fi, use the IP, Ethernet / hotspot
`Connection timed out`	different networks, VPN, guest Wi-Fi	same Wi-Fi, VPN off
`Permission denied`	wrong user or password	retype (case!) or re-flash
`HOST IDENTIFICATION HAS CHANGED`	I re-flashed the card	`ssh-keygen -R raspi4.local`
red LED, green never flickers	no OS found on the card	reseat, re-flash, other card
no LEDs at all	no power	other cable / charger (5 V / 3 A)

my notes

23. Two Pis, two laptops

- Office laptop + home laptop = two **windows** into a Pi. Code and notes live on the Pi and in my GitHub repo. `raspi4.local` only works inside one network.





- **Pass 1:** learn on one Pi (about 2 hours). **Pass 2:** repeat on the other Pi from my notes only.
- Different host names (`raspi-office` , `raspi-home`), same user, `git pull` everywhere.

my notes

QUIZ & WRAP-UP

24. Quick quiz (answer here first!)

1. What is the difference between cloud AI and Edge AI?
2. Name three advantages of running AI at the edge.
3. Image classification: fixed-function or generative? Give one example of the other kind.
4. Name three things an operating system manages.
5. Why enable SSH in Imager *before* writing the card?
6. Red LED steady = ? Green LED flickering = ?
7. What can go wrong if I unplug the Pi without shutting it down?
8. Which command shows the IP address? Which shows whether the OS is 64-bit?

my notes

25. My connection card

hostname	
user name	
password hint (not the password!)	
Wi-Fi 1 / Wi-Fi 2	
IP address (it can change)	
OS release / Python version	
uname -m / microSD size	
date flashed	

my notes

26. Day 1 is done when...

- [ ] the Pi boots: red LED steady, green flickers
- [ ] PowerShell shows `piuser@raspi4:~ $`
- [ ] all commands ran and I wrote down OS + Python
- [ ] I shut down with `sudo shutdown -h now`
- [ ] the connection card is filled in

my notes

Tomorrow, Day 2: remote access like a pro (files, remote desktop, two networks).